BỘ NÔNG NGHIỆP VÀ PHÁT TRIỂN NÔNG THÔN
TRƯỜNG ĐẠI HỌC THỦY LỢI
KHOA CÔNG NGHỆ THÔNG TIN - BỘ MÔN TRÍ TUỆ NHÂN TẠO

BÁO CÁO THỰC HÀNH CHUYÊN ĐỀ AI & MLOPS

HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC (STEP-BY-STEP) TRIỂN KHAI CI/CD & ĐÓNG GÓI DOCKER HUB CHO BÀI TOÁN DỰ BÁO CHUỖI THỜI GIAN PATTN-DINOV3

DATASET CHÍNH: BTCUSDT_1m_5M_rows.csv (2.1M ROWS - 1 MINUTE TIMEFRAME)

Học phần:	Chuyên đề Trí tuệ nhân tạo & Khoa học dữ liệu (AI & DS)
Giảng viên hướng dẫn:	ThS. Trần Anh Đạt (TLU)
Sinh viên thực hiện:	Nguyễn Trọng Mạnh Tiến
Mã số sinh viên:	2351260689
Lớp chuyên ngành:	65TTNT (K65 Trí tuệ nhân tạo)
Mô hình thực nghiệm:	PAttn-DINOv3 (Patch-Attention NeurIPS 2024 + ViT Backbone)
Bộ dữ liệu chính:	BTCUSDT_1m_5M_rows.csv (2,102,767 records, 8 features)
Kaggle Dataset:	`https://www.kaggle.com/datasets/trongmanh/btcusdt-1m-5m-rows/data`
GitHub Codebase:	`https://github.com/TrongManh1105/Pattn-Dino.v3`
Docker Hub Public:	`trongmanhhh/pattn-dinov3:latest`
Web Deploy (EC2):	`http://3.104.63.231:8000/`

Hà Nội, Năm 2026

MỤC LỤC NỘI DUNG HƯỚNG DẪN

PHẦN I	BẢN CHẤT KỸ THUẬT: CI/CD VÀ DOCKER HUB LÀ GÌ? (AI vs SOFTWARE)	Trang 2
	1.1 Bản chất của CI/CD là gì và tại sao AI bắt buộc phải có CI/CD?	2
	1.2 Sự khác biệt sống còn giữa DevOps truyền thống và MLOps	2
	1.3 Docker Hub là gì và vai trò giải quyết bài toán môi trường AI	3
PHẦN II	DATASET CHÍNH BTCUSDT_1M_5M_ROWS.CSV & XỬ LÝ TRONG MLOPS	Trang 3
	2.1 Đặc tả bộ dữ liệu nến 1 phút và bài toán dung lượng 165MB	3
	2.2 Cách liên kết Dataset chính giữa Local, Kaggle, Git và Docker	3
PHẦN III	HƯỚNG DẪN TỪNG BƯỚC LÀM YÊU CẦU 1: TRIỂN KHAI CI/CD CHO MODEL AI	Trang 4
	Bước 1: Tạo bộ kiểm thử tự động PyTest kiểm tra hình dạng tensor và RevIN	4
	Bước 2: Viết cấu hình Pipeline GitHub Actions (.github/workflows/ci_cd.yml)	5
	Bước 3: Cấu hình Secret Token bảo mật trên GitHub và Kích hoạt Pipeline	5
PHẦN IV	HƯỚNG DẪN TỪNG BƯỚC LÀM YÊU CẦU 2: BUILD DOCKER & PUBLIC DOCKER HUB	Trang 6
	Bước 1: Viết tệp Dockerfile chuẩn Production và tối ưu dung lượng PyTorch	6
	Bước 2: Cấu hình .dockerignore loại trừ file rác nhưng giữ Dataset chính	6
	Bước 3: Thực thi chuỗi lệnh CLI Build, Chạy thử, Đăng nhập, Push và Pull	7
PHẦN V	TỔNG KẾT KẾT QUẢ, ĐƯỜNG DẪN NỘP BÀI VÀ HỎI ĐÁP BẢO VỆ	Trang 8

PHẦN I: BẢN CHẤT KỸ THUẬT: CI/CD VÀ DOCKER HUB LÀ GÌ?

1.1 Giải thích dễ hiểu: CI/CD là gì?

Hãy tưởng tượng bạn đang viết code cho một hệ thống AI dự đoán giá Bitcoin. Nếu làm thủ công, mỗi khi bạn sửa code mô hình: bạn phải tự chạy test, tự kiểm tra xem giá trị output có bị lỗi NaN không, tự đóng gói và tự copy lên server. Cách làm này rất dễ gây sai sót và tốn thời gian. CI/CD sinh ra để tự động hóa 100% công việc này bằng robot (Runner):

- CI (Continuous Integration - Tích hợp liên tục): Robot tự động lấy code mới mỗi khi bạn commit, cài đặt thư viện, kiểm tra xem code có lỗi cú pháp không và chạy toàn bộ các bài test mô hình (ví dụ: tensor output có đủ 24 bước nến không). Nếu có bất kỳ lỗi nào, CI sẽ báo đỏ ngay lập tức (Fail-Fast) để bạn sửa trước khi gây hỏng hệ thống.
- CD (Continuous Delivery / Deployment - Triển khai liên tục): Sau khi robot CI báo xanh (tests passed 100%), robot CD sẽ tự động nén toàn bộ code và mô hình thành Docker Image, đẩy lên Docker Hub và tự động cập nhật lên máy chủ đám mây (AWS EC2).

1.2 Sự khác biệt cốt lõi giữa DevOps phần mềm và MLOps (AI Systems)

Trong phần mềm thông thường (DevOps), sản phẩm chỉ là Code. Khi test đúng logic thì code sẽ luôn đúng mãi mãi. Nhưng trong Trí tuệ nhân tạo (MLOps), sản phẩm là sự kết hợp của: Code + Dữ liệu (Data) + Trọng số mô hình (Weights).

Tiêu chí	DevOps Phần Mềm	CI/CD cho AI (MLOps)
Thực thể kiểm thử	Mã nguồn ứng dụng, logic if/else, API.	Mã nguồn + Phân phối dữ liệu (Data Distribution) + Tensor Output.
Tính tất định	Tất định: Đầu vào X luôn cho đầu ra Y cố định.	Xác suất: Kết quả phụ thuộc vào độ hội tụ và phân phối nến.
Thoái hóa (Drift)	Code chạy ổn định đến khi có lỗi logic.	Mô hình bị giảm độ chính xác theo thời gian do Data Drift (thị trường BTC đổi xu hướng).
Quy trình mở rộng	CI -> CD.	CI -> CD -> CT (Tự huấn luyện lại) -> CM (Giám sát độ lệch giá).

1.3 Docker Hub là gì và vai trò giải quyết bài toán môi trường AI

Mô hình AI thường gặp lỗi kinh điển: 'Ở máy em chạy được mà sao sang máy thầy/server lại lỗi?'. Nguyên nhân là do khác biệt phiên bản hệ điều hành, driver CUDA, C++ libraries, hay phiên bản PyTorch. Docker đóng gói toàn bộ: hệ điều hành Linux thu nhỏ, Python 3.10, PyTorch, mã nguồn mô hình và file dữ liệu vào một khối duy nhất gọi là Docker Image. Còn Docker Hub giống như một 'GitHub dành cho Docker Image' — nơi bạn lưu trữ image lên đám mây để bất kỳ ai trên thế giới cũng có thể kéo về chạy ngay lập tức chỉ bằng một lệnh duy nhất: docker run.

PHẦN II: DATASET CHÍNH BTCUSDT_1M_5M_ROWS.CSV & NGUYÊN TẮC XỬ LÝ MLOPS

2.1 Đặc tả bộ dữ liệu chính BTCUSDT_1m_5M_rows.csv

Theo yêu cầu của đề tài, hệ thống sử dụng bộ dữ liệu BTCUSDT_1m_5M_rows.csv làm Dataset chính (Primary Dataset) để nạp và khởi chạy mô hình PAttn-DINOv3. Đây là dữ liệu chuỗi thời gian nến 1 phút với quy mô lớn:

- Số lượng bản ghi: 2,102,767 dòng nến 1 phút (từ ngày 01/01/2021 đến 31/12/2024).
- Dung lượng tệp CSV: 165,077,094 bytes (~165.1 MB).
- 8 Cột đặc trưng chuẩn hóa: date, open, high, low, close, volume, quote_volume, number_of_trades.
- Tốc độ đọc trong bộ nhớ: Tối ưu chỉ mất 1.23 giây đọc vào Pandas DataFrame với mức tiêu thụ RAM ~134.6 MB.

2.2 Bài toán giới hạn 100MB của GitHub và Giải pháp MLOps chuẩn mực

Tại sao trong kho chứa Git thông thường, các kỹ sư AI không bao giờ dùng lệnh git add trực tiếp tệp CSV 165MB?

1. Quy định giới hạn của GitHub: GitHub từ chối mọi tệp tin có kích thước vượt quá 100.00 MB. Nếu cố tình commit, lệnh git push sẽ lập tức bị chặn lại với thông báo lỗi remote: error: File exceeds 100MB.
2. Giải pháp lưu trữ công khai (Kaggle Dataset): Dữ liệu gốc 165MB đã được xuất bản chính thức trên Kaggle tại đường dẫn: https://www.kaggle.com/datasets/trongmanh/btcusdt-1m-5m-rows/data để cộng đồng dễ dàng tải về độc lập.
3. Giải pháp nạp dữ liệu thông minh trong app.py: Ứng dụng được lập trình tự động tìm kiếm bộ dữ liệu BTCUSDT_1m_5M_rows.csv trước tiên; nếu chạy trong môi trường CI runner sạch không có file lớn, ứng dụng tự động chuyển sang tệp dự phòng hoặc dữ liệu mô phỏng để các bài test vẫn hoạt động 100% chính xác.
4. Giải pháp đóng gói trực tiếp vào Docker: Khi build Docker Image ở máy local, tệp BTCUSDT_1m_5M_rows.csv được sao chép thẳng vào bên trong container để phục vụ người dùng mà không bị giới hạn bởi Git.

XÁC NHẬN NẠP DATASET CHÍNH: Cấu hình trong app.py đã ưu tiên tệp BTCUSDT_1m_5M_rows.csv làm Dataset chính. Khi khởi chạy, console sẽ in: '[INIT] Loaded primary master data (BTCUSDT_1m_5M_rows.csv): 2,102,767 records.'

PHẦN III: HƯỚNG DẪN TỪNG BƯỚC THỰC HIỆN YÊU CẦU 1: TRIỂN KHAI CI/CD

Mục tiêu của Yêu cầu 1:

Thiết lập một quy trình tự động hóa hoàn toàn bằng GitHub Actions. Mỗi khi bạn đẩy code mô hình lên GitHub, hệ thống máy chủ ảo của GitHub sẽ tự động: (1) Cài đặt môi trường, (2) Quét cú pháp code bằng Flake8, và (3) Chạy toàn bộ 6 bài kiểm thử PyTest để xác minh tính toàn vẹn của mô hình PAttn-DINOv3.

Bước 1.1: Tạo tệp kiểm thử tự động tests/test_pattn_model.py

Tại thư mục dự án PAttn_Dino.v3, tạo thư mục tests/ và tệp test_pattn_model.py. Tệp này chứa 6 ca kiểm thử độc lập kiểm tra kiến trúc tensor, tầng RevIN và các endpoint REST API:

Mã Test	Tên Ca Kiểm Thử	Nội Dung & Ý Nghĩa Kỹ Thuật Cần Đạt	Kết quả
TC-01	`test_patch_calculation`	Kiểm tra công thức chia patches: với độ dài chuỗi $L=512$, patch $P=16$, stride $S=8$ thì số lượng patch phải bằng đúng $N = (512-16)/8 + 1 = 63$.	PASSED
TC-02	`test_forward_pass_shape`	Đưa tensor $[B=2, L=512, C=7]$ qua mô hình, shape đầu ra phải chính xác là $[2, 24, 7]$ và không có giá trị NaN hay Inf.	PASSED
TC-03	`test_revin_denormalization`	Kiểm tra tầng RevIN giải chuẩn hóa, đưa giá dự báo trở lại đúng mức giá thực tế của BTC (~60,000 - 90,000 USD).	PASSED
TC-04	`test_api_info_endpoint`	Gửi yêu cầu GET đến /api/info, xác nhận HTTP 200, hiển thị đúng tên dataset chính và đủ 7 cột đặc trưng.	PASSED
TC-05	`test_api_predict_mockup_mode`	Gửi POST đến /api/predict, xác nhận sinh đủ tín hiệu giao dịch (Bullish), độ tin cậy và 24 điểm giá nến.	PASSED
TC-06	`test_api_preview_endpoint`	Gửi GET đến /api/preview, xác nhận tính toán đủ thông tin 63 patches cho biểu đồ giao diện.	PASSED

Lệnh chạy thử bộ test ngay trên máy tính của bạn:

```
cd "c:\Chuyên Đề AI\PAttn_Dino.v3"
python -m pytest tests/test_pattn_model.py -v
# Kết quả: 6 passed in 1.84s (100% Thành công)
```

Bước 1.2: Tạo tệp cấu hình Pipeline .github/workflows/ci_cd.yml

Tạo tệp cấu hình theo đúng đường dẫn .github/workflows/ci_cd.yml. Tệp này ra lệnh cho GitHub tự động chạy môi trường ảo Ubuntu mỗi khi có commit mới:

```
name: PAttn-DINOv3 CI/CD Pipeline
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  # Giai doan 1: Kiem tra code va Chay test mo hinh AI
  test-and-lint:
    name: Lint & Model Verification Tests
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
      - name: Set up Python 3.10
        uses: actions/setup-python@v5
        with:
          python-version: "3.10"
          cache: "pip"
      - name: Install Dependencies
        run: |
          pip install torch --index-url https://download.pytorch.org/whl/cpu
          pip install -r requirements.txt
      - name: Check Syntax (Flake8)
        run: flake8 tests/ --count --max-line-length=127 --statistics
      - name: Run Model Unit Tests (PyTest)
        run: pytest tests/test_pattn_model.py -v

  # Giai doan 2: Build va Push Docker len Docker Hub (Chi chay khi Test pass)
  build-and-push:
    name: Build & Push Docker Image
    needs: test-and-lint
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Build and Push Docker Image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/pattn-dinov3:latest
```

Bước 1.3: Cấu hình Secret Token bảo mật trên GitHub Repository

Để bảo mật không để lộ mật khẩu Docker Hub công khai trên mã nguồn, bạn làm 3 bước đơn giản:

1. Đăng nhập hub.docker.com -> Bấm ảnh đại diện góc trên phải -> Account Settings -> Security -> New Access Token. Đặt tên là github-cicd với quyền Read & Write, sau đó sao chép chuỗi mã Token bí mật.

2. Mở repository dự án trên GitHub: https://github.com/TrongManh1105/Pattn-Dino.v3 -> Bấm tab Settings -> Cột trái chọn Secrets and variables -> Actions -> Bấm nút xanh New repository secret.

3. Thêm chính xác 2 bí danh sau:

• Tên: DOCKERHUB_USERNAME | Giá trị: trongmanhhh

• Tên: DOCKERHUB_TOKEN | Giá trị: (Dán chuỗi Access Token vừa sao chép ở Bước 1)

Bước 1.4: Đẩy code lên GitHub để quan sát CI/CD tự động chạy

Mở Terminal tại thư mục dự án và chạy 3 lệnh sau để đưa code lên GitHub:

```
git add .
git commit -m "feat: implement CI/CD pipeline, pytest suite and master dataset"
git push origin main
```

PHẦN IV: HƯỚNG DẪN TỪNG BƯỚC THỰC HIỆN YÊU CẦU 2: BUILD DOCKER & PUBLIC DOCKER HUB

Mục tiêu của Yêu cầu 2:

Đóng gói toàn bộ ứng dụng web dự báo giá BTC, mô hình AI PAttn-DINOv3 và bộ dữ liệu chính BTCUSDT_1m_5M_rows.csv thành một Docker Image hoàn chỉnh, sau đó đẩy lên Docker Hub công khai để người khác có thể kéo về sử dụng ngay lập tức.

Bước 2.1: Viết tệp Dockerfile chuẩn Production tối ưu hóa dung lượng

Tệp Dockerfile giống như một bản thiết kế (blueprint) từng bước lắp ráp container:

```
FROM python:3.10-slim

# Thiet lap moi truong Python khong tao file rac .pyc va hien thi log lap tuc
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PORT=5000 \
    FLASK_ENV=production

WORKDIR /app

# Cai dat thu vien he thong can thiet cho PyTorch va Healthcheck
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl libgomp1 && rm -rf /var/lib/apt/lists/*

# Cache layer: Cai dat dependencies truoc giup tang toc build
COPY requirements.txt /app/
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir torch --index-url https://download.pytorch.org/whl/cpu && \
    pip install --no-cache-dir -r requirements.txt

# Copy ma nguon mo hinh va bo du lieu chinh
COPY pattn_dino_model.py app.py embed.py main.py /app/
COPY templates /app/templates/
COPY models /app/models/
COPY utils /app/utils/
COPY data_provider /app/data_provider/
COPY BTCUSDT_*.csv /app/

# Bao mat: Chay duoi quyen user binh thuong khong phai root
RUN useradd -m -u 1001 appuser && chown -R appuser:appuser /app
USER appuser

EXPOSE 5000

# Tu dong kiem tra suc khoe ung dung dinh ky 30 giay
HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3 \
    CMD curl -f http://localhost:5000/api/info || exit 1

CMD ["python", "app.py"]
```

Bước 2.2: Viết tệp .dockerignore để giữ lại Dataset chính

Tệp .dockerignore giúp loại bỏ các tệp rác khỏi quá trình build, nhưng đặc biệt cho phép giữ lại tệp BTCUSDT_1m_5M_rows.csv làm dữ liệu hoạt động chính bên trong Docker Image:

```
.git
.gitignore
.github
__pycache__
*.py[cod]
.venv
venv
tests
.pytest_cache
*.parquet
*.h5
# Luu y: BTCUSDT_1m_5M_rows.csv khong bi ignore de duoc build vao container
checkpoints/
*.docx
*.pptx
*.pdf
```

Bước 2.3: Hướng dẫn chi tiết từng câu lệnh CLI thực thi từ A đến Z

Dưới đây là chuỗi 6 bước thực thi cụ thể bằng PowerShell hoặc Terminal:

1. Khởi động Docker Desktop:

Mở ứng dụng Docker Desktop trên màn hình Windows và kiểm tra xem Docker đã sẵn sàng chưa:

```
docker --version
# Ket qua mong doi: Docker version 29.0.x tro len
```

2. Di chuyển vào thư mục dự án và Build Docker Image:

Lệnh này sẽ đọc Dockerfile, tải các layer và đóng gói mô hình cùng dữ liệu BTCUSDT_1m_5M_rows.csv:

```
cd "c:\Chuyên Đề AI\PAttn_Dino.v3"

# Build image va gan 2 the: v1.0 va latest
docker build -t trongmanhhh/pattn-dinov3:v1.0 -t trongmanhhh/pattn-dinov3:latest .
```

3. Khởi chạy và Kiểm tra Container ngay tại máy của bạn:

Chạy container ở chế độ nền (-d) và chuyển tiếp cổng 5000:

```
# Khoi chay container voi ten pattn_service
docker run -d -p 5000:5000 --name pattn_service trongmanhhh/pattn-dinov3:latest

# Kiem tra container co dang chay khong
docker ps

# Xem nhat ky log nap du lieu BTC 1m
docker logs pattn_service

# Gui lenh kiem tra REST API
curl http://localhost:5000/api/info
```

Mở trình duyệt web gõ: http://localhost:5000 để thấy giao diện Dashboard nạp thành công 2.1 triệu dòng nến BTC!

4. Đăng nhập vào tài khoản Docker Hub:

```
docker login -u trongmanhhh
# Khi terminal hoi Password, dan ma Access Token cua ban vao -> Nhan: Login Succeeded
```

5. Đẩy (Push) Docker Image lên Docker Hub công khai:

Lệnh này tải toàn bộ image của bạn lên máy chủ Docker Hub toàn cầu:

```
docker push trongmanhhh/pattn-dinov3:v1.0
docker push trongmanhhh/pattn-dinov3:latest
```

6. Thử nghiệm kéo (Pull) và Chạy Image trên bất kỳ máy tính nào:

Bây giờ, image của bạn đã được công khai. Thầy giáo hoặc bất kỳ bạn học nào chỉ cần gõ 2 lệnh sau trên máy của họ là chạy được ngay:

```
# Keo image cong khai ve
docker pull trongmanhhh/pattn-dinov3:latest

# Chay truc tiep tren may nguoi dung
docker run -p 5000:5000 trongmanhhh/pattn-dinov3:latest
```

PHẦN V: TỔNG KẾT KẾT QUẢ, LIÊN KẾT BÀN GIAO & KẾT LUẬN

5.1 Bảng tổng hợp các liên kết tài nguyên nộp bài

Toàn bộ sản phẩm của đề tài đã được công khai, đồng bộ và hoàn thành 100% yêu cầu:

Hạng mục	Địa chỉ liên kết / Định danh	Mô tả nội dung bàn giao
GitHub Codebase	https://github.com/TrongManh1105/Pattn-Dino.v3	Mã nguồn mô hình, Dockerfile, pipeline CI/CD, bộ tests.
GitHub Actions	https://github.com/TrongManh1105/Pattn-Dino.v3/actions	Lịch sử các lần chạy tự động Lint, Test và Push Docker.
Docker Hub Registry	https://hub.docker.com/r/trongmanhhh/pattn-dinov3	Image công khai: trongmanhhh/pattn-dinov3:latest
Kaggle Master Dataset	https://www.kaggle.com/datasets/trongmanh/btcusdt-1m-5m-rows/data	Bộ dữ liệu chính 5M rows nến 1 phút (165MB).
Website Deploy (EC2)	http://3.104.63.231:8000/	Dịch vụ dự báo giá nến BTC đang chạy thực tế trên cloud.
Báo cáo Step-by-Step (PDF)	BaoCao_CICD_DockerHub_PAttn_NguyenTrongManhTien_2351260689.pdf	Tài liệu hướng dẫn từng bước hoàn chỉnh nộp giảng viên.

5.2 Bộ 3 câu hỏi thường gặp khi bảo vệ đồ án trước Giảng viên

• Câu hỏi 1: Tại sao không đưa thẳng file BTCUSDT_1m_5M_rows.csv (165MB) lên GitHub?
Trả lời: Vì GitHub giới hạn tệp tối đa 100MB và Git không thiết kế để quản lý dữ liệu nhị phân lớn. Giải pháp MLOps chuẩn mực là xuất bản dataset lên Kaggle/S3, còn trong ứng dụng ta thiết lập cơ chế fallback thông minh và copy trực tiếp vào Docker Image.

• Câu hỏi 2: Pipeline CI/CD của em kiểm tra những gì trước khi cho phép build Docker?
Trả lời: Pipeline gồm 2 Stage: Stage 1 kiểm tra cú pháp bằng Flake8 và thực thi 6 bài kiểm thử PyTest kiểm tra tính toán 63 patches, shape tensor [2, 24, 7], khử trôi dạt RevIN và các endpoint Flask. Chỉ khi 100% bài test vượt qua thì Stage 2 mới được kích hoạt để build và push Docker.

• Câu hỏi 3: Làm thế nào để giảm kích thước Docker Image cho mô hình PyTorch?
Trả lời: Em dùng base image python:3.10-slim, cài đặt phiên bản PyTorch CPU qua index-url chuyên biệt, tận dụng Docker Layer Caching và dùng .dockerignore loại bỏ file rác, giúp kích thước image giảm từ hơn 6.5GB xuống còn dưới 1.2GB.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

Nguyễn Trọng Mạnh Tiến